

AR525 · IIT Mandi · Reinforcement Learning in Robotics

MC-PILOT Throwing Robotic Arm

A Complete Project Guide — Architecture, Learning Path,
Known Issues, and a Roadmap to Generalization

Reproducing and extending

Turcato et al., “*Data-Efficient Robotic Object Throwing with Model-Based Reinforcement Learning*”
arXiv:2502.05595 (MC-PILOT)

Compiled from the repository's README, `paper/change_history.md`, `paper/current_config.md`,
the LaTeX report sections, and the upstream MC-PILCO codebase.

Generated 2026-07-10.

Contents

1. What This Project Does
2. How It Works — Architecture & Algorithm
3. Repository Map — What Each Variant Adds
4. The Two Convergence Rules (and why they exist)
5. Canonical Hyperparameters & Why They Differ From the Paper
6. How to Learn This Project (a study path)
7. Where to Learn More (references, mapped to files)
8. What's Wrong — Bugs Found, Fixed, and Still Latent
9. Known Limitations (as stated by the project itself)
10. What Can Be Improved
11. How to Make This More General
12. Command Cheat Sheet

1. What This Project Does

This repository is a course project (AR525, IIT Mandi) that reproduces and then extends **MC-PILOT** (Turcato et al., arXiv:2502.05595) — a model-based reinforcement-learning algorithm that teaches a robot arm to throw a ball into a target bin from roughly **10 real trials**. The headline claim of the original paper, and the thing this project sets out to test and stretch, is extreme data efficiency: instead of thousands of throws (model-free RL) or hundreds (residual-physics + deep nets, e.g. TossingBot), MC-PILOT converges from single digits by combining a probabilistic dynamics model with a Monte-Carlo policy gradient and firing the policy only once per throw.

The reproduction lives in `mc-pilot/` (a pure NumPy ballistic simulator, ground-level targets). Five further studies extend it along independent axes, each in its own self-contained directory:

Study	Directory	What it investigates	Headline result
Baseline	<code>mc-pilot/</code>	Reproduce MC-PILOT in a NumPy ballistic sim	5/5 hits, cost 0.74 → 0.001 in 5 trials
1 — Elevated release	<code>mc-pilot-elevated/</code>	Release heights $z = 1.0 / 1.5 / 2.0$ m, stratified exploration	10/10 hits at every height
2 — Multi-arm + noise	<code>mc-pilot-pybullet/</code>	Full PyBullet arm physics (KUKA iiwa7, Franka Panda, xArm6); velocity-slip and salt-and-pepper noise	100% hit rate on commanded velocity across all arms; noise-aware policy beats naive by up to 5× (100% vs. 0% hit rate under 20% slip)
3 — PyBullet + elevated	<code>mc-pilot-pb-elevated/</code>	Combines real arm physics with elevated targets	10/10 hits (Config B, $z = 1.0$ m)
4 — Wind	<code>mc-pilot-wind/</code>	Constant wind, gusts, Ornstein–Uhlenbeck turbulence; blind (2-D) vs. wind-aware (4-D) policy	Counterintuitive: the <i>blind</i> policy wins at 15 trials — see §8
5 — Vision	<code>mc-pilot-pybullet-yolo/</code>	OpenCV HSV segmentation + YOLOv8n bin detection feeding the target estimate	End-to-end detect → throw pipeline; YOLOv8n mAP@0.5 > 0.95 on synthetic data

The vendored `MC-PILCO/` directory is the unmodified upstream reference implementation (Mitsubishi Electric Research Labs, AGPL-3.0) that the whole project is forked from — it is treated as read-only library code throughout.

2. How It Works — Architecture & Algorithm

2.1 The three-module loop

Every variant follows the same orchestration, driven by a subclass of the upstream `MC_PILCO` class (`policy_learning/MC_PILCO.py`):

1. **Model learning** (`model_learning/Model_learning.py`) — a `Ballistic_Model_learning_RBF`: three independent sparse Gaussian Processes, one per velocity-change component (Δv_x , Δv_y , Δv_z). Input is the 6-D ball state `[x, y, z, vx, vy, vz]` (target dimensions are stripped before they reach the GP). Trained per-trial on all trajectory data collected so far, via Adam on the marginal log-likelihood.
2. **Policy** (`policy_learning/Policy.py`) — a `Throwing_Policy`: an RBF network mapping a 2-D target `(Px, Py)` to a single scalar release speed, squashed into `[0, uM]` via $(uM/2) \cdot (\tanh(x)+1)$. Crucially, this policy fires **once**, at the instant of release — not at every timestep — which is the single biggest reason 5–10 trials suffice.
3. **Cost function** (`policy_learning/Cost_function.py`) — a Gaussian distance-to-target cost with lengthscale `lc`, used both to score trials and to compute the policy gradient.

2.2 State augmentation and geometry-conditioning

The ball's physical state $\mathbf{x} = [\mathbf{p}, \dot{\mathbf{p}}] \in \mathbb{R}^6$ is concatenated with the target $\mathbf{P} = (P_x, P_y)$ to form an 8-D augmented state $\tilde{\mathbf{x}} = [\mathbf{x}, \mathbf{P}] \in \mathbb{R}^8$. The GP is trained on this augmented state, so it learns *geometry-conditioned* residual dynamics corrections rather than a single fixed trajectory — one model generalizes across all targets in the domain instead of needing to be retrained per target.

2.3 Physics propagation

State reconstruction from the learned GP output follows a midpoint-rule integration (paper Eq. 18), which is more accurate than forward Euler:

$$\mathbf{v}(t+1) = \mathbf{v}(t) + \Delta \mathbf{v}(t) \quad | \quad \mathbf{p}(t+1) = \mathbf{p}(t) + T_s \cdot \mathbf{v}(t) + (T_s/2) \cdot \Delta \mathbf{v}(t)$$

Aerodynamic drag (paper Eq. 35) is applied as $\mathbf{F}_D = -\frac{1}{2}\rho \cdot C_D(\text{Re}) \cdot A \cdot \|\dot{\mathbf{p}}\| \cdot \dot{\mathbf{p}}$; under wind (Study 4), the relative velocity $\mathbf{v}_{\text{rel}} = \dot{\mathbf{p}} - \mathbf{w}$ is substituted throughout so drag opposes motion through the air mass, not motion over ground.

The physics backend differs per variant: `simulation_class/model.py` is a pure-NumPy ODE integrator (baseline, elevated, wind); `model_pybullet.py` drives a full PyBullet rigid-body arm + ball simulation via `robot_arm/arm_controller.py` (inverse kinematics + a three-phase joint trajectory: neutral → windup → release); `model_mujoco.py` exists upstream but is unused by this project's studies.

2.4 The Monte Carlo policy gradient (the “MC” in MC-PILOT)

This is the part that distinguishes MC-PILOT/MC-PILCO from the original PILCO algorithm, and it is the conceptual core of the whole codebase. Instead of propagating a single analytic Gaussian belief through the GP dynamics via moment matching (as PILCO does), MC-PILCO propagates **M = 400 stochastic particles** through the GP posterior using the reparameterization trick:

$$\mathbf{f}(\tilde{\mathbf{x}}) = \mu(\tilde{\mathbf{x}}) + \mathbf{L}(\tilde{\mathbf{x}}) \cdot \boldsymbol{\varepsilon}^{(m)}, \quad \boldsymbol{\varepsilon}^{(m)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

where μ is the GP posterior mean, $\mathbf{L}$ is the Cholesky factor of the posterior covariance, and each particle's noise sample $\boldsymbol{\varepsilon}$ is drawn once and held fixed — independent of the policy parameters θ — so gradients can flow from the terminal cost, back through every simulated timestep, through the GP, and into θ via ordinary autodiff (`cost.backward()` in PyTorch). The gradient estimate is:

$$\nabla_{\theta} J(\theta) \approx (1/M) \sum_m \nabla_{\theta} c(\tilde{\mathbf{x}}_T^{(m)})$$

optimized with Adam for up to `Nopt = 1500` steps per trial. Gradient variance falls as $O(1/\sqrt{M})$.

Why Monte Carlo instead of PILCO's analytic moment matching?

PILCO's moment matching is only exact when the state distribution stays Gaussian at every step. In the throwing task the initial release velocity is a *nonlinear* function of the target, $\mathbf{v} = (v \cdot \cos\alpha \cdot \cos\phi, v \cdot \cos\alpha \cdot \sin\phi, v \cdot \sin\alpha)$ — so the initial distribution over states is already non-Gaussian at $t = 0$. Moment-matching's Gaussian closure would introduce a systematic bias right from the start; sampling particles instead propagates the true empirical distribution with no distributional assumption.

2.5 What makes MC-PILOT different from generic MC-PILCO

- **Single-shot policy.** The policy is evaluated once, at release, instead of at every control step. This collapses the search space enormously and is the primary reason convergence happens in 5–10 trials instead of the dozens MC-PILCO needs for continuous-control tasks like cartpole swing-up.
- **Gripper-delay estimation.** The original paper's real hardware has a gripper that opens with a delay $t_d \sim U(a, b)$, causing systematic velocity undershoot; the paper estimates (a, b) via Bayesian Optimization on calibration throws and pre-compensates release timing. This project has no physical gripper, so Study 2 substitutes *injected simulated noise* (velocity slip, timing jitter, salt-and-pepper) as a stand-in for that same phenomenon, and lets RL learn the compensation directly instead of using BO. See §8 for why this substitution mattered.

2.6 Algorithmic lineage (from the project's own background section)

Approach	Data needed	Mechanism
Model-free RL	10^3 – 10^4 throws	No dynamics model; pure trial-and-error
TossingBot (2020)	~500 throws	Residual physics + deep net correction
PILCO (2011)	~50 throws	GP dynamics + analytic moment-matching gradient
MC-PILCO (2022)	~20–50 throws	GP dynamics + Monte Carlo gradient
MC-PILOT (2025)	~10 throws	Single-shot policy + gripper-delay BO

3. Repository Map — What Each Variant Adds

Every `mc-pilot*/` directory is a full, self-contained fork of `MC-PILCO/` (own `gpr_lib/`, `simulation_class/`, `policy_learning/`, `model_learning/`). **There is no shared library between variants** — a fix to shared logic (e.g. a GP bug) must be manually ported to every variant that needs it. This is a deliberate but costly design choice; see §10 for the consequences.

Directory	Built from	Key files added
mc-pilot/	MC-PILCO/ (full copy)	simulation_class/model.py → ThrowingSystem ; model_learning/Model_learning.py → Ballistic_Model_learning ; policy_learning/Policy.py → Throwing_Policy + Random_Throwing_Exploration ; Cost_function.py → Throwing_Cost ; MC_PILCO.py → MC_PILOT subclass.
mc-pilot- elevated/	mc-pilot/ (full copy)	Stratified_Throwing_Exploration in Policy.py ; per-height test scripts test_mc_pilot_b.py ... e_strat5.py ; check_max_range.py .
mc-pilot- pybullet/	mc-pilot/ (full copy)	robot_arm/arm_controller.py (IK, Jacobian pseudoinverse, 3-phase trajectory); robot_arm/noise_models.py (VelocityBiasNoise , VelocitySlipNoise , ReleaseTimingJitter , SaltAndPepperVelocityNoise); simulation_class/model_pybullet.py → PyBulletThrowingSystem ; multi-arm scripts for Franka Panda / xArm6; noise-sweep tooling; YOLO dataset/training scripts.
mc-pilot- pb- elevated/	mc-pilot- pybullet/ (noise scripts stripped)	vel_limit_multiplier and base_position kwargs on ArmController / PyBulletThrowingSystem to reach elevated heights within joint-velocity limits; test_mc_pilot_pbe_B/C/D.py .
mc-pilot- wind/	mc-pilot/ baseline (symlinked libs)	simulation_class/wind_models.py (ConstantWind , GustWind , TurbulentWind); model_wind.py → WindThrowingSystem ; Model_learning_wind.py → WindAware_Ballistic_Model_learning_RBF (8-D GP input); WindAware_Throwing_Policy (4-D RBF); MC_PILOT_Wind .
mc-pilot- pybullet- yolo/	mc-pilot- pybullet/ (full copy)	robot_arm/depth_camera.py (RGB-D capture, HSV bin segmentation, depth back-projection, near-face bias correction); vision-integrated demo_pybullet_gui.py with --target-source vision .

4. The Two Convergence Rules (and why they exist)

Two empirically-discovered rules recur across every study and are the single most important transferable insight in this codebase — more important, arguably, than any individual study's result.

4.1 Stratified exploration

Failure mode without it

On `mc-pilot-elevated` Config B ($z = 1.0$ m) with `seed = 1`, plain random exploration produced **0/10 hits**. All throws landed at the same maximum range regardless of target, and cost stayed stuck at 0.70–0.73. Root cause: the random seed happened to draw only low release speeds during the 5 exploration throws, so the GP never observed high-speed dynamics, and the policy gradient found a degenerate “always throw at max speed” local minimum. The same seed with *stratified* exploration converged to 10/10 by trial 3.

The fix, `Stratified_Throwing_Exploration`, partitions $[0, uM]$ into `Nexp` equal speed bands and samples exploration throw i from band i :

$$v_i \sim U((i-1) \cdot uM/Nexp, i \cdot uM/Nexp)$$

This guarantees full-range GP coverage regardless of random seed. Scripts with `_strat` in the name use this; scripts without it can silently fail to converge, and the failure looks like a converged-but-wrong policy, not a crash — which makes it dangerous to skip.

4.2 RBF lengthscale must scale with target range

Failure mode without it

Config E (ground-level, narrow target domain, 0.35 m wide) failed **five times in a row** (E-Strat1 through E-Strat4) under different fixes to exploration strategy, `Nexp`, and `lc` — the policy kept outputting exactly max speed for every target. The real cause, found only on the fifth attempt, was RBF sensitivity: with the paper's default lengthscale `ls = 1.0`, two targets 0.35 m apart produce basis-function activation $\exp(-0.5 \cdot (0.35/1.0)^2) \approx 0.941$ — i.e. the policy network is nearly blind to which target it was given. Every other fix addressed a symptom; this was the actual root cause the whole time.

The rule that fixes it, and generalizes across all five release heights without per-height tuning:

$$S(d, ls) = \exp(-d^2 / 2ls^2) \rightarrow ls^* \approx 0.15 \times \text{target_range}$$

With `ls = 0.15` for Config E, the same two targets now give sensitivity $\exp(-0.5 \cdot (0.35/0.15)^2) \approx 0.066$ — clearly distinguishable — and the config converges to 10/10 with 26 mm mean error.

Practical takeaway

Both rules exist because the paper's defaults were tuned for one specific geometry (a Franka Panda arm, fixed release height, ~1.0 m target range). Neither rule is stated in the original paper; both were discovered by this project through repeated, expensive trial-and-error across five release heights. Anyone porting MC-PILOT to a new geometry should apply both rules *before* debugging anything else.

5. Canonical Hyperparameters & Why They Differ From the Paper

The authoritative table lives in `paper/current_config.md`. Six values were changed from the paper's simulation defaults, each for a documented empirical reason — not arbitrary tuning:

Parameter	This project	Paper (sim)	Why changed
<code>Ts</code>	0.02 s	0.01 s	Doubled for $\sim 3\times$ speedup; confirmed no accuracy regression.
<code>T</code>	0.7 s	1.0 s	Trimmed — ball always lands by ~ 0.58 s; the extra 0.3 s was wasted compute.
<code>lc</code>	0.5 m	0.1 m	At 0.1 m, early exploration errors (0.3–1.5 m) saturate the Gaussian cost to ≈ 1.0 , killing the policy gradient before learning can start. Widening restores usable gradient signal from trial 1.
<code>lm</code>	1.75 m	2.4 m (paper's non-sim value)	2.4 m is beyond the arm's reachable range in this setup; reverted to the paper's own simulation value.
Exploration policy	Random (baseline); Stratified (elevated+)	Near-optimal baseline policy (Eq. 13)	Tested the paper's approach directly — it failed (hit rate 50% $\rightarrow$ 20%): near-optimal throws all look alike and starve the GP of diverse coverage.
Delay modeling	None (numpy sim) / injected noise (PyBullet variants)	Gripper delay <code>t_d</code> $\sim U(a, b)$, fit via Bayesian Optimization	No physical gripper exists in simulation; Study 2's noise models substitute for this.

`Nexp=5`, `Nopt=1500`, `M=400`, `Nb=250`, `uM=3.5 m/s`, `lm=0.75 m`, `gammaM=pi/6` are kept identical to the paper.

6. How to Learn This Project (a study path)

This is written as a sequence — each step assumes the previous one is done. Total time for a motivated reader with basic ML background: roughly one focused week.

Step 0 — Prerequisites

- Gaussian Process regression: what a kernel is, what "posterior mean/covariance" means, why sparse/SOD approximations exist. (Rasmussen & Williams ch. 2 is the standard reference, or any GP intro.)
- Basic policy-gradient RL: what a policy is, what a cost/reward is, why you'd want $\nabla J(\theta)$.

- PyTorch autodiff basics: `.backward()`, why leaf tensors need `requires_grad`, what a broken gradient path looks like (this project hit exactly that bug — see §8).

Step 1 — Read the original papers, in this order

1. **PILCO** (Deisenroth & Rasmussen, 2011) — the GP-dynamics + analytic moment-matching idea.
2. **MC-PILCO** (Amadio et al., *IEEE T-RO* 2022) — replaces moment matching with Monte Carlo particle sampling; this is what `MC-PILCO/` in this repo implements.
3. **MC-PILOT** (Turcato et al., arXiv:2502.05595) — the throwing-specific paper this whole project reproduces: single-shot policy, gripper-delay BO, the 5-throw convergence claim.

Step 2 — Read the upstream reference code (read-only, do not run experiments here)

`MC-PILCO/policy_learning/MC_PILCO.py` → `gpr_lib/GP_prior/Stationary_GP.py` and `Sparse_GP.py` → `simulation_class/model.py`. Goal: understand the generic orchestration class before seeing how it's specialized.

Step 3 — Read the baseline fork end to end

In `mc-pilot/`, in this order: `simulation_class/model.py` (`ThrowingSystem`) → `model_learning/Model_learning.py` (`Ballistic_Model_learning`) → `policy_learning/Policy.py` (`Throwing_Policy`) → `policy_learning/Cost_function.py` (`Throwing_Cost`) → `policy_learning/MC_PILCO.py` (`MC-PILOT` — pay close attention to `apply_policy()`, this is where the single-shot policy and the GP rollout are wired together) → `test_mc_pilot.py` (the entry point). Then actually run it:

```
cd mc-pilot/ && python test_mc_pilot.py -seed 1 -num_trials 10
```

Expect 5/5 hits by trial 5, cost 0.74 → 0.001. Watch the printed per-trial cost — this is the ground truth you'll compare every later change against.

Step 4 — Read `paper/change_history.md` in full

This is the single most valuable document in the repository. It is a real, warts-and-all lab notebook: every bug, every failed experiment, every root-cause diagnosis, in chronological order. Reading the "Experiments That Did NOT Work" subsection for the baseline (Experiments A–G) is a faster way to internalize the two convergence rules (§4) than reading the final config alone — you see *why* each rule was necessary, not just that it exists.

Step 5 — Work through the studies in increasing complexity

1. `mc-pilot-elevated/` — same algorithm, different geometry. Confirms the lengthscale/exploration rules generalize.
2. `mc-pilot-pybullet/` — swaps the physics backend for a real rigid-body arm simulation and introduces the noise-injection layer. Read `robot_arm/arm_controller.py` and `noise_models.py`,

then `paper/sections/study3_noise.tex` for Theorem 1 (why zero-mean noise is unlearnable).

3. `mc-pilot-pb-elevated/` and `mc-pilot-wind/` can be read in either order — both are combinations/extensions of what's already understood by this point.
4. `mc-pilot-pybullet-yolo/` last — it depends on understanding the PyBullet arm pipeline first.

Step 6 — Read the discussion and conclusion

`paper/sections/discussion.tex` and `conclusion.tex` synthesize all five studies into general principles (see §9 of this guide, which reproduces them). This is the payoff step — it reframes everything read so far as instances of a few general lessons about geometry sensitivity and noise structure.

7. Where to Learn More (references, mapped to files)

Topic	Best source in-repo	External reference
What MC-PILOT is / results summary	Root <code>README.md</code>	arXiv:2502.05595
Every design decision + why, chronologically	<code>paper/change_history.md</code>	—
Canonical hyperparameters	<code>paper/current_config.md</code>	arXiv:2502.05595 Table 1
GP regression / sparse GP math	<code>gpr_lib/GP_prior/Stationary_GP.py</code> , <code>Sparse_GP.py</code>	Rasmussen & Williams, <i>Gaussian Processes for Machine Learning</i>
Monte Carlo policy gradient theory	<code>paper/sections/background.tex</code>	Amadio et al. 2022, <i>IEEE T-RO</i> 38(6)
Formal proof: zero-mean noise is unlearnable	<code>paper/sections/study3_noise.tex</code> (Theorem 1)	—
Wind / curse-of-dimensionality result	<code>paper/sections/study4_wind.tex</code> , <code>mc-pilot-wind/final_analysis.txt</code>	—
Vision pipeline design	<code>paper/sections/study5_vision.tex</code> , <code>mc-pilot-pybullet-yolo/TASK_CHANGELOG_DEPTH_CAMERA_BIN.md</code>	Ultralytics YOLOv8 docs
Formal limitations statement	<code>paper/sections/discussion.tex</code>	—
Original up-front design (pre-implementation)	<code>old/implementation_plan.md</code>	—
Compiled full report	<code>paper/Group_3_Report.pdf</code>	—

8. What's Wrong — Bugs Found, Fixed, and Still Latent

The project's own change log is unusually candid about mistakes. This section is organized as a bug list because that is the most useful way to internalize the codebase's actual failure surface — these are exactly the mistakes a newcomer extending this code is likely to repeat.

8.1 Bugs found and fixed (baseline)

Bug	Symptom	Root cause	Fix
T_control unit mismatch	"Nearly crashed the laptop"	<code>T_control</code> passed in steps instead of seconds → effective horizon of 10,000 steps	Pass seconds, not <code>T/Ts</code>
<code>opt_steps_list</code> index error	Index out of range	List sized for <code>num_trials</code> only, but the trial-index counter starts at <code>Nexp-1</code>	Size list to <code>Nopt × (Nexp + num_trials)</code>
Broken gradient path	<code>RuntimeError: element 0 of tensors does not require grad</code>	<code>data_to_gp_input()</code> stripped the policy's speed output before it reached the graph	Rewrote <code>apply_policy()</code> to keep speed → velocity conversion differentiable via <code>atan2 / cos / sin</code>
Post-landing GP hallucination	Cost stuck at ~0.998, never improving	Particles kept propagating for all 100 steps even after the ball had already landed, producing physically meaningless "underground" trajectories	Freeze particle state with a <code>landed</code> boolean mask once $z \leq 0$ (paper §4.2.2)
Cost saturation	Cost plateaus ~0.88 no matter what	<code>lc = 0.1</code> saturates the Gaussian cost to ≈ 1.0 for any error > 0.15 m — exactly the error range of early exploration throws	Widen <code>lc</code> to 0.5
Jacobian unpacking	Crash in <code>arm_controller.py</code>	<code>p.calculateJacobian</code> returns 2 values (linear, angular), code unpacked 3	Unpack correctly
Landing-detection false positive	Ball "landing" 3.23 m away (physically wrong)	Landing check used <code>pos[2] <= 0.0</code> , but the ball's center rests at $z \approx \text{ball_radius}$ on the ground plane, so it kept bouncing/rolling before the check ever triggered	Check <code>pos[2] <= ball_radius + 0.005</code>
<code>load_model_from_log</code> truncation	Wrong state history on resumed runs	Bug triggered specifically when <code>num_explorations > 1</code>	Patched inside <code>MC_PILOT_Wind</code>

8.2 The RBF "ghost center" family of bugs

This bug pattern recurred at least twice, independently

Once in the baseline (Experiment F: porting the paper's Eq. 22 center-initialization domain directly caused ~37% of the 250 RBF centers to fall in a dead zone where targets never spawn, corrupting the policy sum), and once far more severely in the wind study (`w1-aware` : wind was constant at 5.0 m/s but the 250 policy centers were initialized uniformly over `[-1.0, 1.0]` — the true wind value was completely outside the center domain, so **every** basis function had near-zero activation and the policy was mathematically dead, producing 0% hit rate and 84 cm mean error). The general lesson: **RBF center/lengthscale initialization must always be derived from the actual data domain of the run being configured, never copied from a different geometry or a different paper table.** This is arguably a more dangerous bug class than a crash, because it fails silently as a converged-looking but non-functional policy.

8.3 Bugs found in the noise study (Study 2)

- **Unlearnable noise chosen by default.** `VelocityBiasNoise` (zero-mean Gaussian) was the first noise model built and tested — but it is provably unlearnable (see Theorem 1, §9). A full experimental cycle was spent on it before this was recognized and the noise models were redesigned around biased (slip, timing-jitter, salt-and-pepper) noise instead.
- **PyBullet position control was reported as ground truth velocity** for `ReleaseTimingJitter`, but joint-limit-induced oscillation made actual end-effector velocity swing from 0.1–6.6 m/s for a 2.0 m/s command — nonsense. Fixed by computing the noise-perturbed velocity analytically instead of reading it back from a poorly-tracked simulated joint.
- **Reachability cap silently capped hit rate at ~75%** — at `alpha=0.12` slip, the aware policy needs commanded speed `v_needed/0.88`, which for the far end of the target range exceeded `um`. The symptom looked like a learning failure; it was a hard physical ceiling. Fixed by raising `um`.
- **GP hyperparameter MLE degeneration.** An experiment isolating pure multiplicative slip (`σ = 0`, no additive noise) produced GP lengthscales of 1791 m, 1602 m, 357 m — i.e. the GP collapsed to fitting a flat line through all training data, because with zero additive noise the likelihood has no regularizing pressure against arbitrarily large lengthscales. **The additive noise term is not just modeling sensor/actuator noise — it is acting as GP regularization**, and removing it entirely breaks the model, not just the "realism" of the noise.

8.4 Latent / acknowledged-but-unfixed issues

- **Train/test mismatch in particle freezing.** During GP rollout training, particles are frozen the instant they cross `z ≤ 0`, but this is not the same as interpolating to the exact landing point the way the deployed simulator does at evaluation time. The project's own defense checklist flags this directly: it introduces roughly 2–6 cm of systematic over-estimation in landing distance during training, which biases cost estimates slightly pessimistically. It has not caused a convergence failure so far, but it is a known and unresolved discrepancy between train-time and test-time physics.
- **Non-monotone anomaly in the noise sweep.** At `alpha_slip = 0.15`, the naive policy achieved 100% hit rate — sandwiched between 25% at 0.10 and 0% at 0.20. The project's own report flags

this as "suspicious" and attributes it to single-seed luck rather than a real effect, but it has not been re-run at additional seeds to confirm.

- **Single-seed reporting throughout.** Most headline numbers in this guide (and in the report) are seed = 1 results. The elevated-Config-B case (0/10 at seed 1 with random exploration vs. 10/10 at seed 2) is direct proof within this project's own results that single-seed numbers can be existence proofs rather than expected performance — yet most of the multi-arm, noise, and elevated results are still reported at a single seed.

9. Known Limitations (as stated by the project itself)

These are drawn verbatim in substance from `paper/sections/discussion.tex` and `paper/report_defense_checklist.md` — i.e. this is the project's own self-assessment, not an external critique.

1. **Simulation only, no hardware validation.** Every result in every study is simulated. The 5–10% PyBullet-vs-NumPy drag discrepancy documented internally is not known to compound or cancel under real-world effects (motor backlash, real sensor noise, real gripper mechanics).
 2. **Statistical coverage is thin.** Most results are single-seed. A ≥ 5 -seed sweep is needed before any hit-rate number here should be read as an expected value rather than a single sample.
 3. **Fixed launch angle ($\alpha = 35^\circ$).** The policy only ever learns release speed; joint optimization of angle and speed together would enlarge the reachable target volume, especially for near-field targets, but was never attempted.
 4. **Wind-aware training was never run at adequate scale.** The study concluded ~ 40 trials are needed for the 4-D wind-aware policy to out-perform the 2-D blind one, but the full-budget experiment that would confirm this was never actually run — it remains a projection from the dimensional-scaling argument ($250^{(1/2)} \approx 15.8$ effective centers/dimension in 2-D vs. $250^{(1/4)} \approx 3.98$ in 4-D), not a measured result.
 5. **Vision pipeline is implemented but not quantitatively evaluated.** Study 5's detect-then-throw pipeline works end-to-end but has not been benchmarked across a randomized target distribution with multi-seed statistics — there is no reported detection-to-throw accuracy number.
 6. **Custom simulator, not the paper's full setup.** The project's own release-height variation is explicitly narrower in scope than "full 3D elevated targets," and the ballistic simulator is a custom implementation, not the original paper's full robot-plus-delay-estimation hardware pipeline.
 7. **Six independent code forks with no shared library.** Any fix to core GP/policy/cost logic must be manually re-applied to every variant it should also affect (see §10).
-

10. What Can Be Improved

This section combines the project's own stated "future work" (from `conclusion.tex`) with structural issues visible in the repository organization itself.

10.1 Directly stated as future work by the project

- Hardware validation on a physical manipulator — this is the single biggest open validity gap.
- Multi-seed statistical evaluation across all five studies (≥ 5 seeds), replacing single-seed anecdotes with confidence intervals / hit-rate distributions.
- A wind-randomized training regime — train one policy across randomly sampled wind conditions per episode, rather than one policy per fixed wind config, to test whether domain randomization sidesteps the curse-of-dimensionality problem found in Study 4 without needing 40+ trials per condition.
- Quantitative end-to-end evaluation of the vision pipeline (Study 5) over a randomized target distribution — currently it "works" in demos but has no accuracy number.

10.2 Structural / engineering improvements not explicitly stated but evident from the codebase

- **Collapse the six forks into one configurable codebase.** The current fork-per-study structure means the same class of bug (e.g. the RBF ghost-center problem, §8.2) had to be independently rediscovered in two different variants. A shared library with variant behavior selected by config (physics backend, noise model, wind model, vision on/off) would let a single fix propagate everywhere and make the two convergence rules (§4) enforceable in one place instead of copy-pasted judgment calls per script.
- **Automate the two convergence rules instead of hand-computing them per config.** Right now, lengthscale and exploration-band width are manually derived per test script from the target range. Both are simple closed-form functions of geometry ($l_s = 0.15 \times \text{target_range}$, N_{exp} bands over $[0, u_M]$) — they could be computed automatically from the declared target domain and arm speed limit at startup, removing an entire category of the "ghost center" bugs in §8.2 by construction.
- **Add regression tests around the fixed bugs.** There is currently no unit-test suite — the project's own docs note "there is no unit test suite... correctness is judged by hit rate / cost convergence." Given how many of the bugs in §8 were silent (converged-looking but wrong), a small regression suite asserting e.g. "seed=1 baseline reaches cost < 0.01 by trial 5" would catch future regressions immediately instead of requiring a human to notice a suspicious plot.
- **Replace $l_c = 0.5$ with a curriculum instead of a single widened constant.** The project found $l_c = 0.1$ starves gradients early (errors too large) but also that $l_c = 0.2$ failed later (Experiment A) once accuracy improved. An annealed l_c — wide during early exploration, narrowing as the policy's accuracy improves — would plausibly get both the early-gradient benefit of a wide cost and the late-training precision of a narrow one, rather than compromising on a fixed 0.5 for the whole run.

- **Confirm or refute the additive-noise-as-regularizer finding (§8.3) more rigorously.** This result (removing additive noise causes GP lengthscale collapse to ~1500+ m) is a genuinely interesting and somewhat alarming finding about the GP's sensitivity to its own noise floor — it deserves a dedicated ablation (sweep σ from 0 upward and plot learned lengthscale) rather than being discovered as a side effect of one debugging session.

11. How to Make This More General

The project already demonstrates several axes of generalization (release height, arm morphology, wind, noise, vision). The natural next steps to make the underlying *algorithm* — not just the benchmark suite — more general are:

11.1 Learn launch angle jointly with speed

Currently $\alpha = 35^\circ$ is fixed and only speed is learned. Extending the policy's output from 1-D (speed) to 2-D (speed, angle) is a direct, well-scoped next step that the project itself flags as missing. This would require re-deriving the RBF output squashing for two bounded outputs and re-checking whether the two convergence rules (§4) still hold with a richer action space — plausibly N_b (basis function count) needs to grow, since a 2-D output surface needs more resolution than a 1-D one for the same lengthscale rule.

11.2 Domain-randomize instead of retraining per condition

Every study here trains one policy per fixed condition (one policy per height, one per wind config, one per arm). A more general system would train a single policy across randomized release heights / wind conditions / noise levels sampled per episode, conditioning the policy on whatever context is observable (target, and — where available — measured wind or arm state). This is exactly the generalization Study 4's "future work" note gestures at, but it's also the natural fix for the "six separate forks" structural problem in §10.2 — a single domain-randomized policy collapses several of today's per-study variants into configurations of one training run.

11.3 Make the GP model uncertainty-aware about extrapolation

Several of the bugs in §8 (ghost centers, ranges beyond reachable targets, GP lengthscale collapse) share a common shape: the model was silently confident in a region it had no real data support for. A general safeguard — checking predictive variance at proposed policy actions during optimization and penalizing high-uncertainty regions in the cost, which is closer to true PILCO/MC-PILCO philosophy than this project's uniform-cost approach — would make future geometry changes fail loudly (as visibly high uncertainty) instead of silently (as a policy that looks converged but is wrong).

11.4 Automatic geometry-to-hyperparameter derivation

As in §10.2: turn the two convergence rules into code that runs at startup given only (target range, uM , arm reach), rather than per-script constants. This is the most direct path to "port MC-PILOT to a

new arm/target geometry without re-deriving lengthscales by hand" — which is currently a multi-day debugging exercise (as §4's failure logs show) and could become an automatic, first-run computation.

11.5 Close the sim-to-real gap deliberately, not just note it

Rather than treating "no hardware validation" purely as a caveat, the noise-injection framework already built in Study 2 (velocity slip, timing jitter, salt-and-pepper) is precisely the right tool to prepare for a real transfer: train under a distribution of injected noise magnitudes (not one fixed α) so the learned policy is robust to whatever the real gripper/arm's true error distribution turns out to be, rather than tuned to one guessed noise level. This turns the existing noise study from a diagnostic experiment into an actual sim-to-real robustness technique.

11.6 Unify the vision pipeline with the noise/uncertainty framework

Study 5's vision pipeline currently feeds a point estimate of the target into the same policy used elsewhere. A more general system would propagate the vision system's detection uncertainty (bounding box confidence, depth noise) into the policy the same way Study 4 propagates wind state — as an additional conditioning input — so the system degrades gracefully under poor lighting/occlusion instead of confidently throwing at a noisy point estimate.

12. Command Cheat Sheet

```
# Environment (Python 3.11 venv used across every variant)
python -m venv .venv
source .venv/bin/activate
pip install torch numpy scipy matplotlib pybullet ultralytics opencv-python

# Baseline
cd mc-pilot/ && python test_mc_pilot.py -seed 1 -num_trials 10

# Study 1 – elevated release (NumPy)
cd mc-pilot-elevated/
python test_mc_pilot_b_strat.py -seed 1 -num_trials 10    # z = 1.0 m
python test_mc_pilot_c_strat.py -seed 1 -num_trials 10    # z = 1.5 m
python test_mc_pilot_d_strat.py -seed 1 -num_trials 10    # z = 2.0 m

# Study 2 – PyBullet arm + noise + multi-arm
cd mc-pilot-pybullet/
python test_mc_pilot_pb_A.py -seed 1 -num_trials 10
python test_mc_pilot_pb_A_noisy.py -seed 1 -num_trials 10 -alpha 0.20
python train_mc_pilot_pb_A_franka_panda.py -seed 1
python run_pb_noise_paper_multiseed.py
python demo_pybullet_gui.py --log_path results_mc_pilot_pb_A/1

# Study 3 – PyBullet + elevated
cd mc-pilot-pb-elevated/ && python test_mc_pilot_pbe_B.py -seed 1 -num_trials 10

# Study 4 – wind
cd mc-pilot-wind/
python run_all_wind_experiments.py --num_trials 15
python analyze_wind_results.py

# Study 5 – vision
cd mc-pilot-pybullet-yolo/
python test_mc_pilot_pb_A.py -seed 1 -num_trials 10
python demo_pybullet_gui.py --log_path results_mc_pilot_pb_A/1 --num_throws 5
python generate_yolo_target_dataset.py && python train_yolo_target_detector.py
```

All test scripts take `-seed` and `-num_trials`. Results land in `results_<config_name>/<seed>/` inside the variant directory. There is no unit-test suite — correctness is judged by hit rate / cost convergence printed to stdout and logged under `results_*/`.

End of guide. Sourced from the repository's README.md, paper/change_history.md, paper/current_config.md, paper/sections/*.tex, old/implementation_plan.md, and the MC-PILCO upstream codebase.